

Wilsy OS Compliance Knowledge Base Playbook

Document class: Production knowledge base / operator playbook

System: Wilsy OS / legal-doc-system

Owner: Wilson Khanyezi

Scope: Account Command Center compliance evidence rail, Git/CI workflow, runtime proof, regulator readiness, login/environment recovery, competitive moat.

Current sealed runtime receipt: ACR_20260622095237520_9238e9f9930747eb

Current live status: LIVE API COMPLIANCE RAIL IS SEALED

Table of Contents

1. Executive Doctrine
2. Current Sealed State
3. Non-Negotiable Completion Criteria
4. System Architecture
5. Compliance Evidence Model
6. Live Runtime Verification
7. Evidence Generation and Regulator Bundle Procedure
8. Database Routing and Environment Awareness
9. Account Command Center UI Rail Requirements
10. Git and CI Operating Discipline
11. Login and Environment Recovery Runbook
12. Incident Playbooks
13. Regulator Evidence Operating Model
14. Competitive Moat and Boardroom Narrative
15. Operator Checklists
16. Appendices

1. Executive Doctrine

Wilsy OS does not claim compliance because a browser component says so. Wilsy OS claims compliance readiness only when the backend can prove it with tenant-scoped, sealed, receipt-backed evidence.

The compliance system is designed around five principles:

1. **Backend authority only.** The browser renders proof but never creates proof.
2. **Tenant-scoped evidence.** Each evidence receipt is bound to a tenant identity.
3. **Sealed receipts.** A compliance state is not sealed until the database contains a receipt with status: SEALED.

4. **Merkle anchoring.** Evidence hashes and Merkle roots allow the system to prove continuity and tamper resistance.

5. **Regulator export readiness.** A sealed compliance rail must be capable of producing a regulator bundle.

The operating position is simple:

```
Competitors show dashboards.  
Wilsy OS shows evidence.
```

```
Competitors show badges.  
Wilsy OS seals receipts.
```

```
Competitors ask for trust.  
Wilsy OS produces proof.
```

2. Current Sealed State

The live runtime API has returned the following final state:

```
Tenant: wilsy-sovereign-root  
Proof source: account_compliance_evidence_receipts  
Receipt count: 1  
Sealed receipt count: 1  
Clauses anchored: 4  
Receipt seal status: SEALED  
Regulator bundle ready: true  
Browser authority: false  
Backend authority: true  
Blockers: []  
Evidence promoted to proof: true
```

2.1 Current sealed receipt

```
Receipt ID:  
ACR_20260622095237520_9238e9f9930747eb  
  
Status:  
SEALED  
  
Generated at:  
2026-06-22T09:52:37.520Z  
  
Evidence hash:  
9238e9f9930747eb1626b83e96b7de5158aa1f4057b6dd4757676d29dc689bb40  
a699bf8d7c6b6b77fe4d79cb8697bc8688502d921388ed28433eae63e3856b9  
  
Merkle root:  
9238e9f9930747eb1626b83e96b7de5158aa1f4057b6dd4757676d29dc689bb40  
a699bf8d7c6b6b77fe4d79cb8697bc8688502d921388ed28433eae63e3856b9
```

2.2 Current rail card state

The live API rail returned:

```
Sealed evidence ledger live  
Receipt-backed audit channel live  
Custody receipt sealed  
Regulator bundle ready  
Tenant evidence ledger sealed
```

3. Non-Negotiable Completion Criteria

The Account Command Center compliance rail is complete only when the live backend endpoint returns all of the following:

```

ok: true
proof.source: account_compliance_evidence_receipts
proof.receiptCount >= 1
proof.sealedReceiptCount >= 1
proof.clausesAnchored >= 1
proof.browserAuthority: false
proof.backendAuthority: true
proof.blockers: []
proof.receiptSealStatus: SEALED
proof.latestReceiptId: present
evidence.receiptCount >= 1
evidence.sealedReceiptCount >= 1
evidence.receiptSealStatus: SEALED
evidence.regulatorBundleReady: true
diagnostics.noFakeData: true
diagnostics.evidencePromotedToProof: true

```

The following states are explicitly **not complete**:

```

NO_RECEIPTS_YET
receiptCount: 0
sealedReceiptCount: 0
blockers not empty
browserAuthority: true
regulatorBundleReady: false
UI-only proof
Fake proof data
Unverified database route

```

4. System Architecture

4.1 Primary components

Layer	Component	Purpose
Frontend	Account Command Center	Renders compliance posture and proof rail
API route	/api/account/compliance-command	Returns runtime compliance state
Service	accountComplianceCommandService	Aggregates posture, services, proof, and rail cards
Evidence service	accountEvidenceCommandService	Reads and promotes receipt-backed evidence
Model	AccountComplianceEvidenceReceipt	Stores sealed compliance receipts
Case model	Case	Supports case/compliance context
Generator	wilsy-generate-account-compliance-evidence.js	Creates sealed receipts and regulator bundle output
Export area	server/exports/regulator-bundles/ /	Runtime regulator bundle files; not source code

4.2 Authority flow

```
User opens Account Command Center
|
v
Frontend requests compliance command API
|
v
Backend resolves tenant and compliance posture
|
v
Backend reads AccountComplianceEvidenceReceipt collection
|
v
If sealed receipts exist, evidence is promoted to proof
|
v
API returns rail cards and actions with backendAuthority true
|
v
Browser renders proof but does not author proof
```

4.3 No fake data rule

When the evidence ledger is empty, the system must return explicit empty state:

```
receiptCount: 0
sealedReceiptCount: 0
receiptSealStatus: NO_RECEIPTS_YET
regulatorBundleReady: false
```

The system must not fabricate:

```
receipt IDs
Merkle roots
sealed statuses
regulator bundle readiness
backend proof state
```

5. Compliance Evidence Model

5.1 Receipt structure

A sealed account compliance receipt contains:

```
receiptId
tenantId
status
evidenceHash
merkleRoot
previousMerkleRoot
complianceBindings
generatedAt
```

5.2 Compliance bindings

Current receipt bindings:

Framework	Control	Meaning
POPIA	privacy-command	Privacy posture is backed by sealed evidence
FICA	audit-defensibility	Audit posture is anchored to evidence
AUDIT	chain-of-custody	Custody is backend-owned and receipt-backed

Framework	Control	Meaning
REGULATOR_EXPORT	regulator-access	Evidence can be packaged for regulator access

5.3 Proof promotion

The API may promote evidence to proof only when:

```
evidence.receiptCount >= 1
evidence.sealedReceiptCount >= 1
evidence.receiptSealStatus == SEALED
evidence.regulatorBundleReady == true
```

When promoted, diagnostics should include:

```
evidencePromotedToProof: true
evidenceProofSource: account_compliance_evidence_receipts
```

6. Live Runtime Verification

6.1 Live API probe

Use this command while the backend is running:

```
cd /Users/wilsonkhanyezi/legal-doc-system

curl -sS
  "http://127.0.0.1:5050/api/account/compliance-command?tenantId=wilsy-sovereign-root&limit=250"
\
-H "X-Tenant-Id: wilsy-sovereign-root" \
| python3 -m json.tool \
| tee /tmp/wilsy-compliance-command-proof-final.json
```

6.2 Seal checker

```
python3 - <<'PYCHECK'
import json
from pathlib import Path

path = Path("/tmp/wilsy-compliance-command-proof-final.json")
data = json.loads(path.read_text())

proof = data.get("proof", {}) or {}
evidence = data.get("evidence", {}) or {}
actions = data.get("actions", {}) or {}

receipt_count = max(int(proof.get("receiptCount") or 0), int(evidence.get("receiptCount") or 0))
sealed_count = max(int(proof.get("sealedReceiptCount") or 0),
    int(evidence.get("sealedReceiptCount") or 0))
clauses = max(int(proof.get("clausesAnchored") or 0), int(evidence.get("clausesAnchored") or 0))
seal_status = evidence.get("receiptSealStatus") or proof.get("receiptSealStatus")
bundle_ready = bool(evidence.get("regulatorBundleReady") or proof.get("regulatorBundleReady"))
backend_authority = proof.get("backendAuthority") is True or actions.get("backendAuthority") is
    True
browser_authority_false = proof.get("browserAuthority") is False or
    actions.get("browserAuthority") is False
blockers_empty = proof.get("blockers", []) == []

checks = {
    "backendAuthority": backend_authority,
    "browserAuthorityFalse": browser_authority_false,
    "blockersEmpty": blockers_empty,
    "receiptCount": receipt_count >= 1,
    "sealedReceiptCount": sealed_count >= 1,
    "clausesAnchored": clauses >= 1,
    "receiptSealStatus": seal_status == "SEALED",
    "regulatorBundleReady": bundle_ready,
}

print("\nWILSY LIVE API COMPLIANCE SEAL CHECK")
print("=====")
for key, ok in checks.items():
    print(f"{'PASS' if ok else 'FAIL'}: {key}")

print("\nEvidence snapshot:")
print(f"receiptCount: {receipt_count}")
print(f"sealedReceiptCount: {sealed_count}")
print(f"clausesAnchored: {clauses}")
print(f"receiptSealStatus: {seal_status}")
print(f"regulatorBundleReady: {bundle_ready}")

if all(checks.values()):
    print("\nFINAL: LIVE API COMPLIANCE RAIL IS SEALED")
else:
    print("\nFINAL: API still not returning sealed runtime evidence")
PYCHECK
```

Expected output:

```
FINAL: LIVE API COMPLIANCE RAIL IS SEALED
```

7. Evidence Generation and Regulator Bundle Procedure

7.1 Standard evidence generation

```
cd /Users/wilsonkhanyezi/legal-doc-system

node scripts/wilsy-generate-account-compliance-evidence.js \
  --tenant=wilsy-sovereign-root \
  --actor=wilson-khanyezi \
  --reason=production-account-command-center-compliance-seal
```

7.2 Generate against the server runtime database

Use this when root .env and server/.env point to different MongoDB databases.

```
cd /Users/wilsonkhanyezi/legal-doc-system

SERVER_MONGODB_URI="$(python3 - <<'PYMONGO'
from pathlib import Path
values = {}
for line in Path("server/.env").read_text().splitlines():
    if "=" in line and not line.strip().startswith("#"):
        k, v = line.split("=", 1)
        values[k.strip()] = v.strip().strip('\'').strip('"')
print(values.get("MONGODB_URI") or values.get("TEST_MONGODB_URI") or "")
PYMONGO
)"

MONGODB_URI="$SERVER_MONGODB_URI" \
TEST_MONGODB_URI="$SERVER_MONGODB_URI" \
node scripts/wilsy-generate-account-compliance-evidence.js \
--tenant=wilsy-sovereign-root \
--actor=wilson-khanyezi \
--reason=seal-api-runtime-database-for-account-command-center
```

7.3 Regulator bundle output

The generator produces regulator bundles under:

```
server/exports/regulator-bundles/
```

These are runtime artifacts and must not be committed unless a deliberate release policy says otherwise.

8. Database Routing and Environment Awareness

8.1 The observed routing wall

The system encountered this situation:

```
Root .env Mongo route:
local development database
- contained sealed receipts

server/.env Mongo route:
server runtime database
- originally contained zero receipts
```

The live API reads the server runtime database. Therefore, receipts in the root/local database do not seal the live API rail.

8.2 Read-only DB routing forensic

```
cd /Users/wilsonkhanyezi/legal-doc-system

python3 - <<'PYROUTE'
from pathlib import Path
from urllib.parse import urlparse

def parse(path):
    values = {}
    if not path.exists():
        return values
    for line in path.read_text().splitlines():
        if "=" in line and not line.strip().startswith("#"):
            k, v = line.split("=", 1)
            values[k.strip()] = v.strip().strip("'").strip('"')
    return values

def safe(uri):
    if not uri:
        return "[missing]"
    p = urlparse(uri)
    return f"{p.scheme}://{p.hostname}:{p.port if p.port else ''}{p.path}"

for file in [Path(".env"), Path("server/.env")]:
    values = parse(file)
    print(file)
    print("MONGODB_URI:", safe(values.get("MONGODB_URI")))
    print("TEST_MONGODB_URI:", safe(values.get("TEST_MONGODB_URI")))
PYROUTE
```

8.3 Rule

A compliance rail is not live-sealed until the **same database read by the running backend API** contains the sealed receipt.

9. Account Command Center UI Rail Requirements

The Account Command Center must render these proof states from the backend response only:

```
Privacy command
Regulatory ledger
Audit defensibility
Chain of custody
Regulator access
Tenant ledger
```

A sealed state should display language equivalent to:

```
Sealed evidence ledger live
Receipt-backed audit channel live
Custody receipt sealed
Regulator bundle ready
Tenant evidence ledger sealed
```

The browser must never invent proof. The API should expose:

```
browserAuthority: false
backendAuthority: true
```

10. Git and CI Operating Discipline

10.1 Current known-good chain

```
f03ace0 Align super admin threat detection test code
c61e81e Provide CI super admin test password
30f8509 Seal Wilsy OS compliance evidence and command rail
f5f1b89 chore(ci): update upload artifact action from v3 to v4 to clear deprecation failure
```

10.2 Pre-work checkpoint

```
cd /Users/wilsonkhanyezi/legal-doc-system

export GIT_PAGER=cat
export PAGER=cat

git --no-pager status --short
git --no-pager log -3 --oneline --decorate
git ls-remote --heads origin main
```

Expected:

```
status --short has no output
HEAD and origin/main match
```

10.3 Backup rule

Before edits:

```
TS="$(date +%Y%m%d-%H%M%S)"
mkdir -p ".wilsy-backup-vault/$TS"
cp "path/to/file" ".wilsy-backup-vault/$TS/path/to/file"
```

Do not stage:

```
.wilsy-backup-vault/
.local-mongo/
server/exports/
reports/
forensic-fixes/
*.bak
*.baseline
*.backup
*.orig
*.rej
*.broken
*.checkpoint
WILSY_*
```

10.4 Guard commands

For changed JS/JSX/MJS/CJS files:

```
node scripts/wilsy-secret-guard.js path/to/file.js
node scripts/wilsy-documentation-guard.js path/to/file.js
```

Frontend build when relevant:

```
cd client && npm run build
```

10.5 Bad artifact gate

Before commit:

```
git --no-pager diff --cached --name-only | grep -E '(\.bak|\.baseline|\.backup|\.orig|\.rej|\.broken|\.checkpoint|^\.local-mongo|^\.wilsy-|^server/exports|^reports|^forensic-fixes|^WILSY_)'\
&& { echo "BLOCKED: bad artifact staged"; exit 1; } \
|| echo "PASS: staged set is clean"
```

After commit:

```
git show --name-only --pretty="" HEAD | grep -E '(\.bak|\.baseline|\.backup|\.orig|\.rej|\.broken|\.checkpoint|^\.local-mongo|^\.wilsy-|^server/exports|^reports|^forensic-fixes|^WILSY_)'\
&& echo "BLOCKED: bad artifact entered commit" \
|| echo "PASS: commit is clean"
```

10.6 CI policy

Do not use --no-verify as a routine fix. Fix the cause.

When CI fails:

```
Confirm latest commit hash.
Confirm it is not an old failed run.
Read the exact red panel.
Reproduce locally with the exact CI command.
Patch source only.
Run guards.
Stage intentional files only.
Push.
Confirm green.
```

11. Login and Environment Recovery Runbook

11.1 Red-zone rule

Do not overwrite .env or server/.env when the system is working and the user is logged in.

Any recovery script must be:

```
local-only
backed up first
redacted when printed
not committed
not a credential reset unless explicitly requested
```

11.2 JWT validation crash

Symptom:

```
FATAL: Environment validation failed
JWT_SECRET must be at least 32 characters
```

Fix:

```
Restore original env or set a 32+ character JWT_SECRET in the correct local env file.
Do not invent a new login password if the existing login worked.
```

11.3 Restart backend safely

```
cd /Users/wilsonkhanyezi/legal-doc-system/server

unset JWT_SECRET
unset JWT_REFRESH_SECRET
unset FORENSIC_HMAC_KEY
unset ADMIN_EMAIL
unset ADMIN_PASSWORD

npm run dev
```

11.4 Clear stale browser tokens only when needed

```
localStorage.clear();
sessionStorage.clear();
location.reload();
```

11.5 Recovery success definition

Login recovery is complete only when:

```
Backend starts without fatal env validation.
User logs in with original credentials.
Authenticated API routes respond.
Git status has no unintended source changes.
```

12. Incident Playbooks

12.1 CI env wall

Symptom:

```
ADMIN_EMAIL and ADMIN_PASSWORD environment variables are required
```

Meaning:

```
GitHub Actions did not provide required test env.
```

Production-grade fix:

```
Store values in GitHub repository secrets.
Reference them as secrets in .github/workflows/ci.yml.
```

12.2 Super admin auth test wall

Symptom:

```
15/15 TESTS PASSED
Process completed with exit code 1
```

Real cause found in JUnit:

```
expected AUTH_009 to equal AUTH_003
```

Resolution:

SA008 threat detection now expects AUTH_009, which is the middleware threat code.

12.3 Documentation guard wall

Symptom:

FUNCTION_JSDOC_REQUIRED: fullLogin

Resolution:

Add adjacent JSDoc with @function, @description, and @collaboration.

12.4 Runtime DB mismatch wall

Symptom:

Generator shows sealed receipts.
Live API shows NO_RECEIPTS_YET.

Meaning:

Generator wrote to a different database than the server runtime API reads.

Resolution:

Generate evidence using the server runtime MongoDB URI, restart backend, re-probe API.

13. Regulator Evidence Operating Model

13.1 What Wilsy OS can prove

Wilsy OS can prove that a tenant has:

backend-owned compliance proof
browser proof disabled
sealed evidence receipt
Merkle root
compliance bindings
regulator bundle readiness

13.2 What Wilsy OS should not overclaim

Allowed claim:

Wilsy OS provides backend-authoritative, tenant-scoped, receipt-backed compliance evidence infrastructure.

Not allowed:

Wilsy OS guarantees legal compliance for every customer in every jurisdiction.

Reason:

Legal compliance depends on configuration, customer process, legal basis, jurisdiction, data use, and counsel review.

13.3 Regulator Q&A

Regulator question	Wilsy OS answer
Who owns proof?	Backend owns proof; browser authority is false.
Is evidence tenant-scoped?	Yes, receipts include tenant ID.
Is proof sealed?	Yes, when receiptSealStatus is SEALED and Merkle root is present.
Can the UI fabricate compliance?	No, the UI renders backend proof only.
Can evidence be exported?	Yes, when regulatorBundleReady is true.

14. Competitive Moat and Boardroom Narrative

14.1 Core position

Wilsy OS is not a dashboard. It is a sovereign compliance command system.

The moat is not visual polish alone. The moat is defensibility:

```
Backend authority
Receipt-backed evidence
Tenant isolation
Merkle anchoring
Regulator bundle readiness
CI-enforced operating discipline
No fake compliance data
```

14.2 Boardroom proof sequence

1. Log into Wilsy OS.
2. Open Account Command Center.
3. Open Compliance rail.
4. Show sealed rail cards.
5. Run live API proof.
6. Show receipt ID.
7. Show Merkle root.
8. Show regulator bundle readiness.
9. Show CI green.
10. Show Git clean.

14.3 Demo command

```
curl -sS
  "http://127.0.0.1:5050/api/account/compliance-command?tenantId=wilsy-sovereign-root&limit=250"
  \
-H "X-Tenant-Id: wilsy-sovereign-root" \
| python3 -m json.tool
```

14.4 Competitive narrative

```
Competitors sell dashboards.  
Wilsy OS sells defensibility.  
  
Competitors show status.  
Wilsy OS shows evidence.  
  
Competitors rely on admin trust.  
Wilsy OS uses backend authority.  
  
Competitors export reports.  
Wilsy OS seals receipts.  
  
Competitors make claims.  
Wilsy OS produces proof.
```

15. Operator Checklists

15.1 Daily system seal checklist

```
[ ] User can log into Wilsy OS  
[ ] Backend starts without fatal env validation  
[ ] Account Command Center opens  
[ ] Compliance rail loads  
[ ] API returns backendAuthority true  
[ ] API returns browserAuthority false  
[ ] API returns blockers []  
[ ] API returns receiptSealStatus SEALED  
[ ] API returns regulatorBundleReady true  
[ ] Git status is clean
```

15.2 Compliance seal checklist

```
[ ] Create /tmp/wilsy-compliance-command-proof-final.json  
[ ] Run seal checker  
[ ] Confirm backendAuthority PASS  
[ ] Confirm browserAuthorityFalse PASS  
[ ] Confirm blockersEmpty PASS  
[ ] Confirm receiptCount PASS  
[ ] Confirm sealedReceiptCount PASS  
[ ] Confirm clausesAnchored PASS  
[ ] Confirm receiptSealStatus PASS  
[ ] Confirm regulatorBundleReady PASS  
[ ] Final line says LIVE API COMPLIANCE RAIL IS SEALED
```

15.3 Git clean checklist

```
cd /Users/wilsonkhanyezi/legal-doc-system  
  
git --no-pager status --short  
git --no-pager log -4 --oneline --decorate  
git ls-remote --heads origin main
```

Pass criteria:

```
status --short has no output  
HEAD equals origin/main  
GitHub Actions green
```

15.4 Release close checklist

```
[ ] Source guards passed
[ ] Build passed where relevant
[ ] Exact local test passed where relevant
[ ] GitHub Actions green
[ ] Runtime API proof green
[ ] Browser UI reflects backend proof
[ ] Git clean
[ ] Backup vault local only
[ ] No env secrets committed
```

16. Appendices

Appendix A: Suggested repository location

Push this Markdown file to:

```
docs/wilsy-os/WILSY_OS_COMPLIANCE_KNOWLEDGE_BASE_PLAYBOOK.md
```

Optional PDF location if the repository stores static assets:

```
docs/wilsy-os/WILSY_OS_COMPLIANCE_KNOWLEDGE_BASE_PLAYBOOK.pdf
```

Appendix B: Push commands

```
cd /Users/wilsonkhanyezi/legal-doc-system

mkdir -p docs/wilsy-os
cp /path/to/WILSY_OS_COMPLIANCE_KNOWLEDGE_BASE_PLAYBOOK.md docs/wilsy-os/
cp /path/to/WILSY_OS_COMPLIANCE_KNOWLEDGE_BASE_PLAYBOOK.pdf docs/wilsy-os/

git add docs/wilsy-os/WILSY_OS_COMPLIANCE_KNOWLEDGE_BASE_PLAYBOOK.md \
      docs/wilsy-os/WILSY_OS_COMPLIANCE_KNOWLEDGE_BASE_PLAYBOOK.pdf

git --no-pager diff --cached --stat

git commit -m "Add Wilsy OS compliance knowledge base playbook"

git push origin main
```

Appendix C: Final safety gate

```
git show --name-only --pretty="" HEAD | grep -E '(\.bak|\.baseline|\.backup|\.orig|\.rej|\.broke
n|\.checkpoint|^\.local-mongo|^\.wilsy-|^server/exports|^reports|^forensic-fixes|^\WILSY_) ' \
&& echo "BLOCKED: bad artifact entered commit" \
|| echo "PASS: commit is clean"

git --no-pager status --short
```

Final Statement

The Wilsy OS Account Command Center Compliance rail is done when the live API returns sealed backend evidence. The system has reached that state for tenant `wilsy-sovereign-root` with sealed receipt `ACR_20260622095237520_9238e9f9930747eb`.

This playbook exists so the seal can be reproduced, defended, audited, and explained without relying on memory, screenshots, or UI claims.